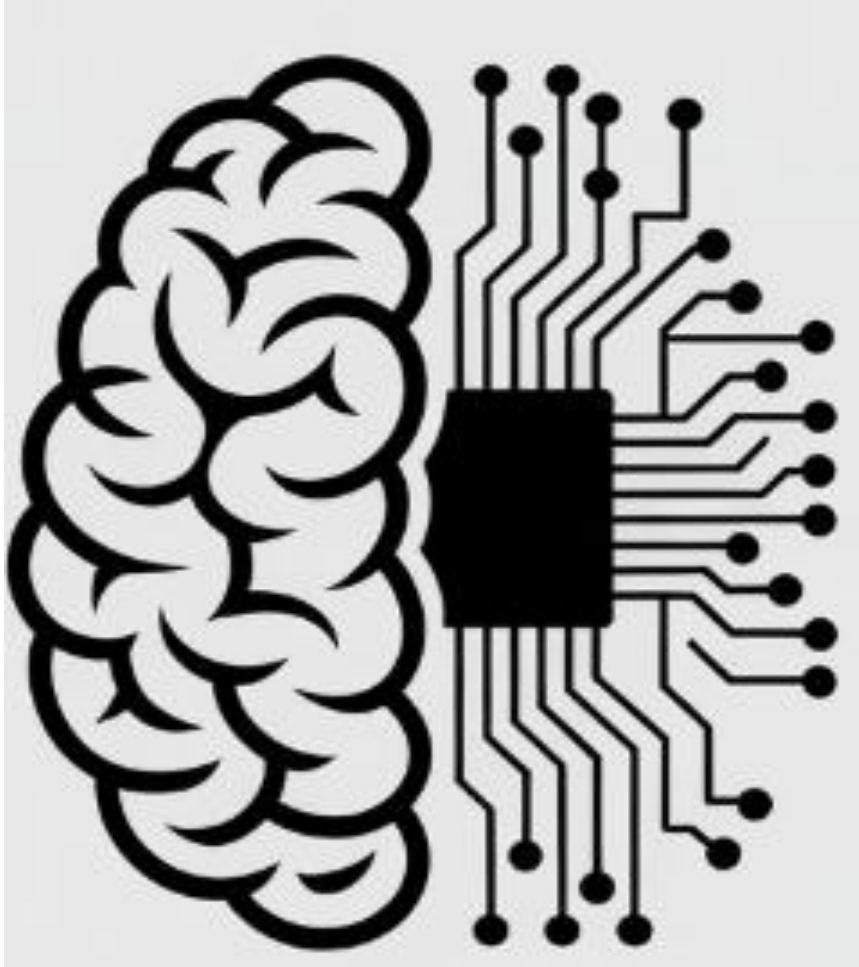




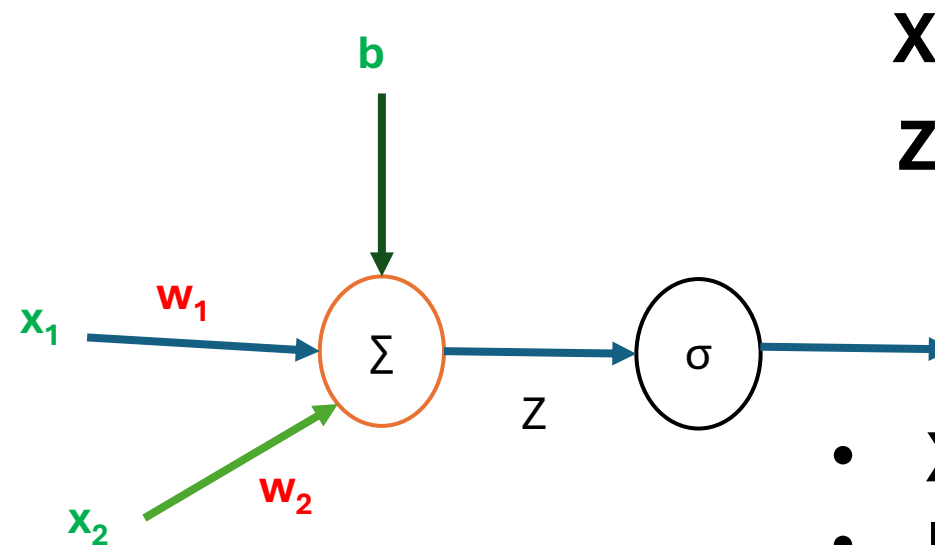
Neural Network

Dr. Sobia Tariq Javed

Suppose we have 2 inputs X_1 and X_2 with bias b .

Perceptron Learning

- **Perceptron Learning** is a supervised learning algorithm used for **binary classification**.
- It is used to classify data that is linearly separable.



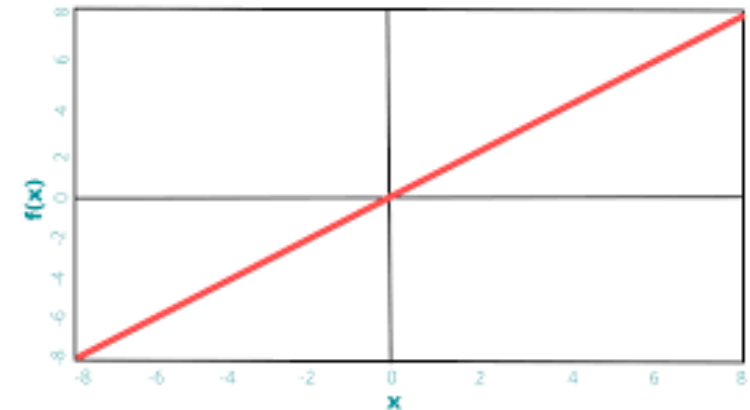
$$X_0 = b$$

$$Z = X_0 + X_1W_1 + X_2W_2$$

- X_1 and X_2 are inputs
- b is called bias
- W_1 and W_2 are weights
- σ is the activation function

Linear Function

- $y = x$
- Linear activation function outputs the same value as the input and does not introduce nonlinearity, making the network behave like a linear model.
- Linear function $\rightarrow$ no activation function



Linear Activation Function

Linear activation Function

Advantages

- **Simple** and **easy** to compute.
- **Fast computation** and **low computational** cost
- Suitable for **regression problems** (predicting continuous values)

Disadvantages

- Cannot learn **complex patterns (no nonlinearity)**
- **Multiple layers** become equivalent to a **single layer**
- Reduces power of **deep neural networks**
- Not suitable for **hidden layers**

Step Function

(a “threshold” or “Heaviside” function)

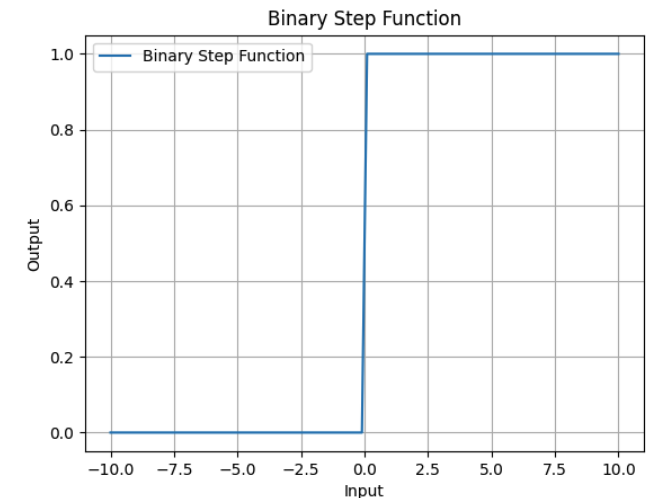
$$f(x) = \begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases}$$

Advantages

- Very simple and computationally inexpensive
- Suitable for early perceptron models
- Clear binary decision boundary

Disadvantages

- Not differentiable at threshold
- Derivative is zero everywhere else
- Not suitable for deep learning



Weights for AND Gate

$$Z = X_0W_0 + X_1W_1 + X_2W_2$$

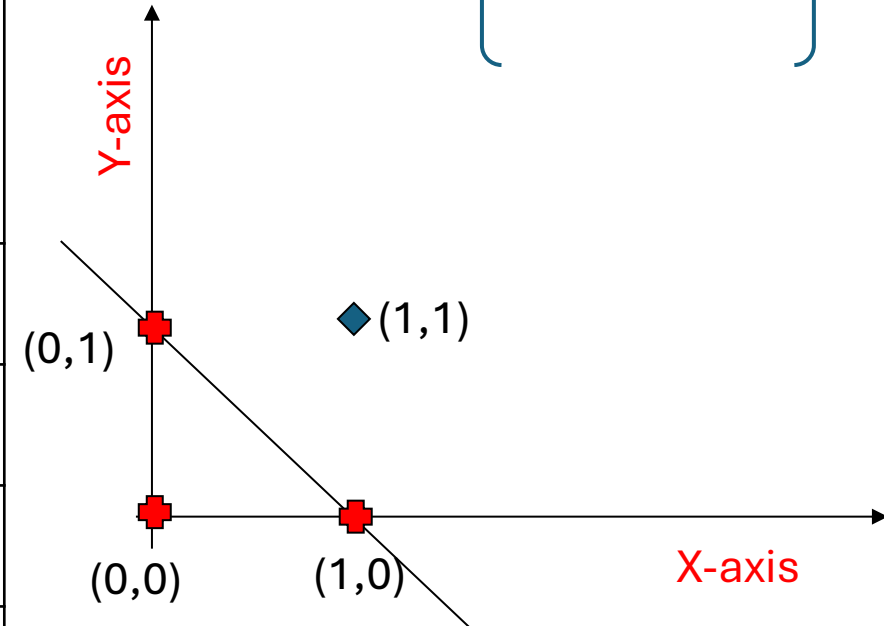
$$X_0=1, \theta=0$$

Find values of W_0 , W_1 and W_2

- A typical AND gate truth table is shown below

Input		Actual Output	Z	Weights	
X_1	X_2				
0	0	0			
0	1	0			
1	0	0			
1	1	1			

$$F = \begin{cases} 1 & \text{if } Z \geq \theta \\ 0 & \text{if } Z < \theta \end{cases}$$



Weights for AND Gate

$$Z = X_0W_0 + X_1W_1 + X_2W_2$$

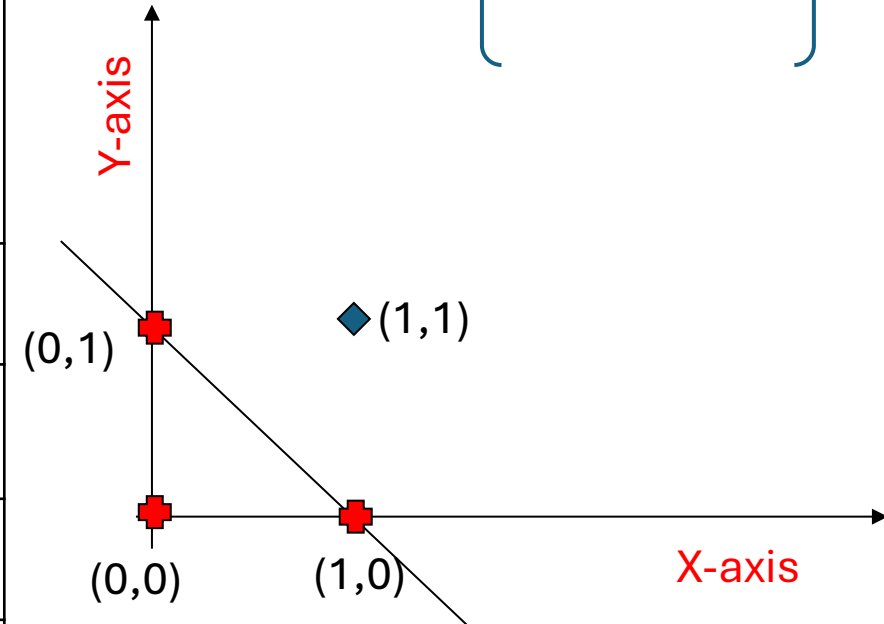
$$X_0=1, \theta=0$$

Find values of W_0 , W_1 and W_2

- A typical AND gate truth table is shown below

Input		Actual Output	Z	Weights	
X_1	X_2				
0	0	0		$W_0 < 0$	
0	1	0		$W_0 + W_2 < 0$	
1	0	0		$W_0 + W_1 < 0$	
1	1	1		$W_0 + W_1 + W_2 \geq 0$	

$$F = \begin{cases} 1 & \text{if } Z \geq \theta \\ 0 & \text{if } Z < \theta \end{cases}$$



$$Z = X_0W_0 + X_1W_1 + X_2W_2$$

$$X_0=1, \theta=0$$

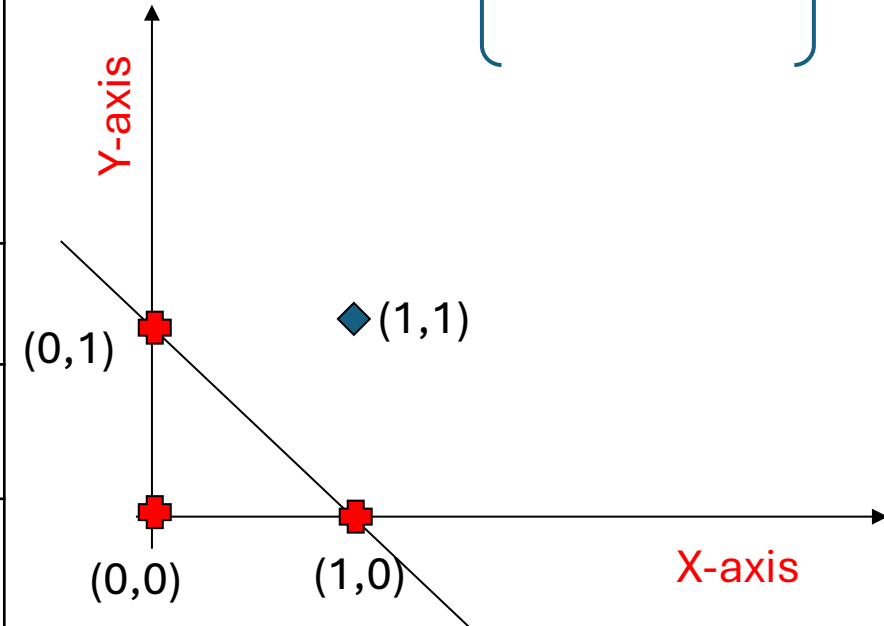
Find values of W_0 , W_1 and W_2

Weights for AND Gate

- A typical AND gate truth table is shown below

Input X_1 X_2		Actual Output	Z	Weights	
0	0	0	-1	$W_0 < 0$	$W_0 = -1$
0	1	0	-0.5	$W_0 + W_2 < 0$	$W_0 = -1, W_2 = 0.5$
1	0	0	-0.5	$W_0 + W_1 < 0$	$W_0 = -1, W_1 = 0.75$
1	1	1	0.25	$W_0 + W_1 + W_2 \geq 0$	$W_0 = -1, W_2 = 0.5, W_1 = 0.75$

$$F = \begin{cases} 1 & \text{if } Z \geq \theta \\ 0 & \text{if } Z < \theta \end{cases}$$



$$Z = X_0W_0 + X_1W_1 + X_2W_2$$

$$X_0=1, \theta=0$$

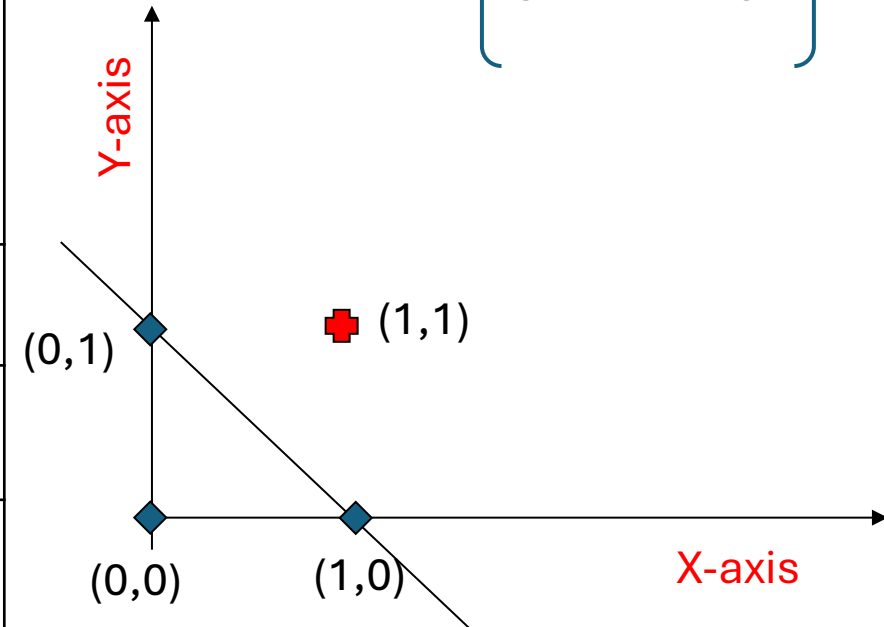
Weights for NAND Gate

Find values of W_0 , W_1 and W_2

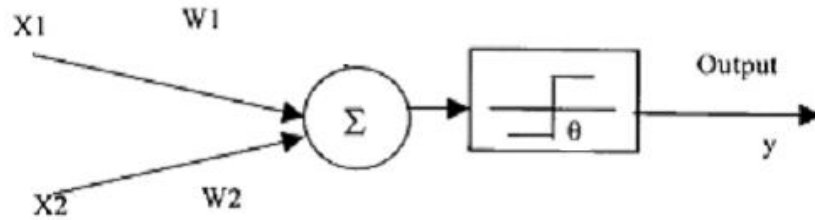
- A typical NAND gate truth table is shown below

Input X_1 X_2		Actual Output	Z	Weights	
0	0	1	1	$W_0 \geq 0$	$W_0 = 1$
0	1	1	0.5	$W_0 + W_2 \geq 0$	$W_0 = 1, W_2 = -0.5$
1	0	1	0	$W_0 + W_1 \geq 0$	$W_0 = 1, W_1 = -1$
1	1	0	-0.5	$W_0 + W_1 + W_2 < 0$	$W_0 = 1, W_2 = -0.5, W_1 = -1$

$$F = \begin{cases} 1 & \text{if } Z \geq \theta \\ 0 & \text{if } Z < \theta \end{cases}$$



OR Gate



$$Z = X_0W_0 + X_1W_1 + X_2W_2$$

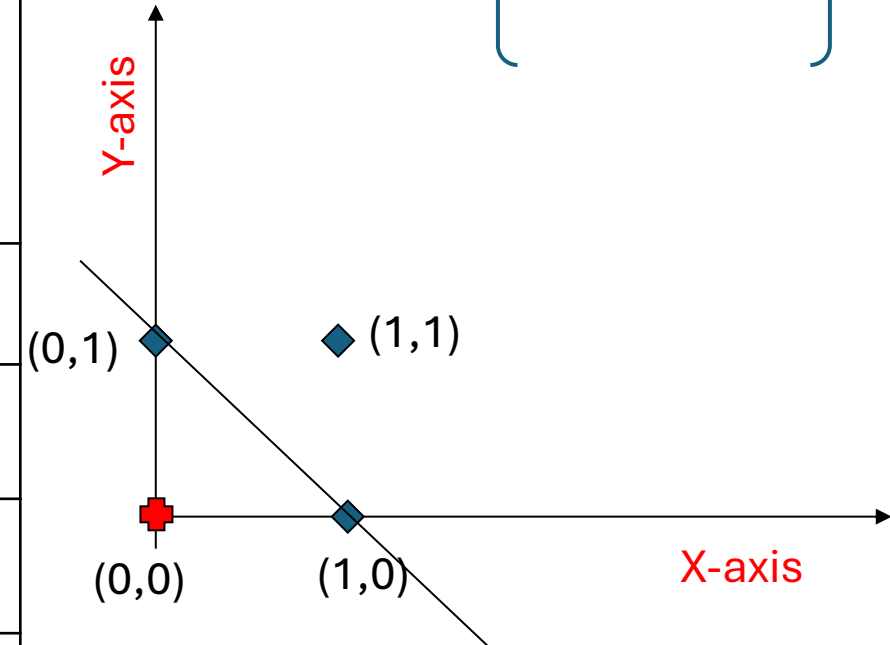
$$X_0=1, \theta=0$$

Find values of W_0 , W_1 and W_2

- A typical OR gate truth table is shown below

Input X_1 X_2		Actual Output	Z	Weights	
0	0	0	-0.5	$W_0 < 0$	$W_0 = -0.5$
0	1	1	0.5	$W_0 + W_2 \geq 0$	$W_0 = -0.5, W_2 = 1$
1	0	1	0.5	$W_0 + W_1 \geq 0$	$W_0 = -0.5, W_1 = 1$
1	1	1	1.5	$W_0 + W_1 + W_2 \geq 0$	$W_0 = -0.5, W_2 = 1, W_1 = 1$

$$F = \begin{cases} 1 & \text{if } Z \geq \theta \\ 0 & \text{if } Z < \theta \end{cases}$$



$$\text{XOR}(X,Y) = (X+Y)(XY)'$$

$$Z = X_0W_0 + X_1W_1 + X_2W_2$$

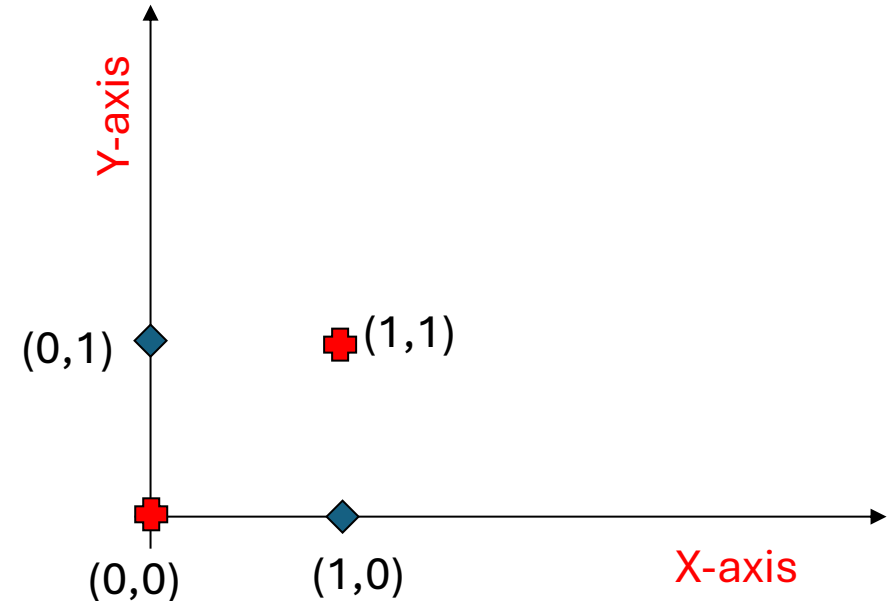
$$X_0=1, \theta=0$$

Step Activation function

XOR Gate

- A typical XOR gate truth table is shown below

Input		Actual Output	Z
X_1	X_2		
0	0	0	
0	1	1	
1	0	1	
1	1	0	



$$\text{XOR}(X,Y) = (X+Y)(XY)'$$

$$Z = X_0W_0 + X_1W_1 + X_2W_2$$

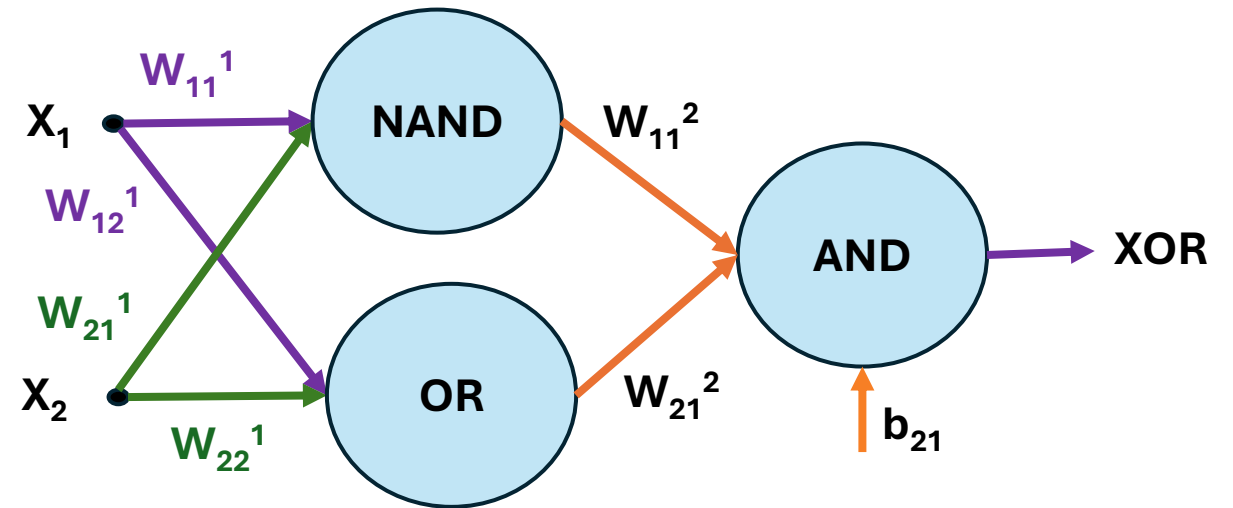
$$X_0=1, \theta = 0$$

Step Activation function

XOR Gate

- A typical XOR gate truth table is shown below

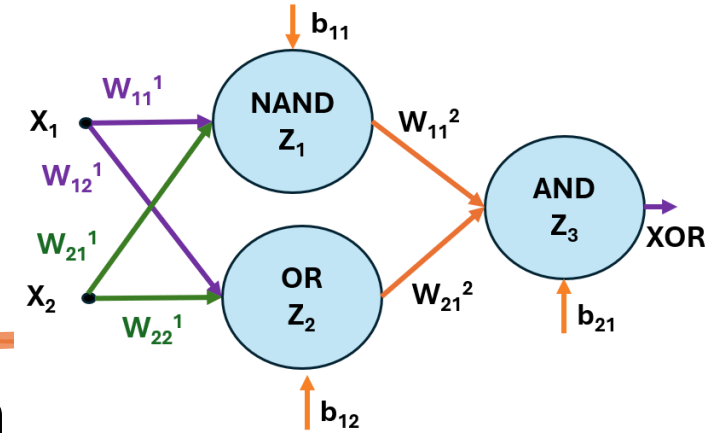
Input		Actual Output	Z
X_1	X_2		
0	0	0	
0	1	1	
1	0	1	
1	1	0	



HW: Find the values of b_{21} , W_{11}^2
And W_{21}^2

$$\text{XOR}(X,Y) = (X+Y)(XY)'$$

XOR Gate



- What if instead of Step function we use linear function

- $Z_1 = b_{11} + X_1 W_{11}^1 + X_2 W_{21}^1$

- $Z_2 = b_{12} + X_1 W_{12}^1 + X_2 W_{22}^1$

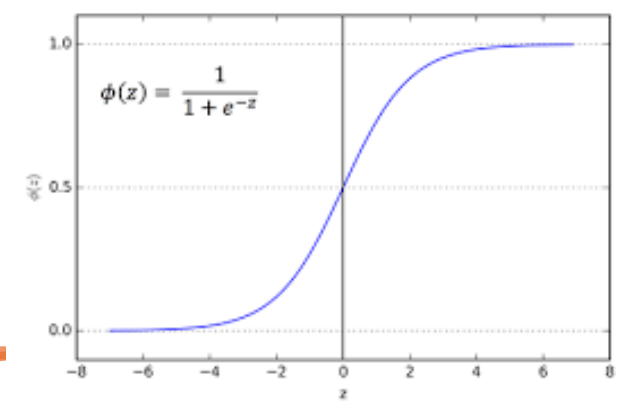
- $Z_3 = [b_{11} + X_1 W_{11}^1 + X_2 W_{21}^1] W_{11}^2 + [b_{12} + X_1 W_{12}^1 + X_2 W_{22}^1] W_{21}^2 + b_{21}$

- $= X_1 \underbrace{[W_{11}^1 W_{11}^2 + W_{12}^1 W_{21}^2]}_A + X_2 \underbrace{[W_{21}^1 W_{11}^2 + W_{22}^1 W_{21}^2]}_B + \underbrace{[b_{11} W_{11}^2 + b_{12} W_{21}^2 + b_{21}]}_C$

- $= X_1 A + X_2 B + C \quad \leftarrow \text{Line}$

- A straight line cannot separate this data
- Linear model fails to classify correctly
- Activation $\rightarrow$ introduces nonlinearity
- Helps the model learn complex and non-linearly separable patterns

Sigmoid function



- $\sigma(x) = \frac{1}{1+e^{-x}}$
- $\frac{d\sigma(x)}{dx} = \frac{1}{1+e^{-x}}$
- $\frac{d\sigma(x)}{dx} = (1 + e^{-x})^{-1}$
- $\sigma'(x) = (-1)(1 + e^{-x})^{-2}(-1)(e^{-x})$
 $= \frac{e^{-x}}{(1+e^{-x})^2}$
 $= \left[\frac{1}{1+e^{-x}} \right] \left[\frac{e^{-x}}{1+e^{-x}} \right]$
 $= \left[\frac{1}{1+e^{-x}} \right] \left[\frac{-1+1+e^{-x}}{1+e^{-x}} \right]$

- $\sigma'(x) = \left[\frac{1}{1+e^{-x}} \right] \left[1 - \frac{1}{1+e^{-x}} \right]$
- Since $\sigma(x) = \frac{1}{1+e^{-x}}$
- $\sigma'(x) = \sigma(x)[1 - \sigma(x)]$

Delta Rule

- Delta rule **updates weights** proportionally to the **error**, helping the model learn by minimizing the difference between **predicted and target outputs**.
- $W_i^{\text{new}} = W_i^{\text{old}} + \Delta W_i$
- $\Delta W_i = \text{learning rate} * (\text{Actual output} - \text{predicted output}) * x_i$
- $W_i^{\text{new}} = W_i^{\text{old}} + \eta(y - \hat{y})x_i$
- It can only learn **linearly separable patterns**.
- It is used with **linear neurons (Perceptron) or (single-layer networks)**.

Gradient Descent

- Gradient descent is an iterative method that **updates parameters** in the **direction of the negative gradient** to minimize the **loss function**.
- $W_i^{\text{new}} = W_i^{\text{old}} - \Delta W_i$
- $\Delta W_i = \text{learning rate} * \text{gradient of Error/loss function w.r.t weight}$
- $W_i^{\text{new}} = W_i^{\text{old}} - \eta * \frac{\partial E}{\partial W_i}$
- Its general optimization method (used in many algorithms).

Delta Rule vs Gradient Descent

- The **delta rule** is a special case of **gradient descent** when applied to a **linear neuron with squared error**.
- **Delta rule** updates **weights using error** directly, while **gradient descent uses the derivative** of the **loss function**.

Derivative vs Gradient

- A **derivative** gives the rate of change of a function with respect to **one variable**, while a **gradient** is a vector that gives the rate of change with respect to **multiple variables**.

$$\frac{dy}{dx}$$

Example: $y = x^2 \Rightarrow \frac{dy}{dx} = 2x$

$$\nabla f = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \dots \right)$$

Example: $f(x, y) = x^2 + y^2 \Rightarrow \nabla f = (2x, 2y)$

- Change in x will bring how much change in y?

$$\frac{dy}{dx} = (+ve) \text{ 2?}$$

$$\frac{dy}{dx} = (-ve) \text{ 2?}$$

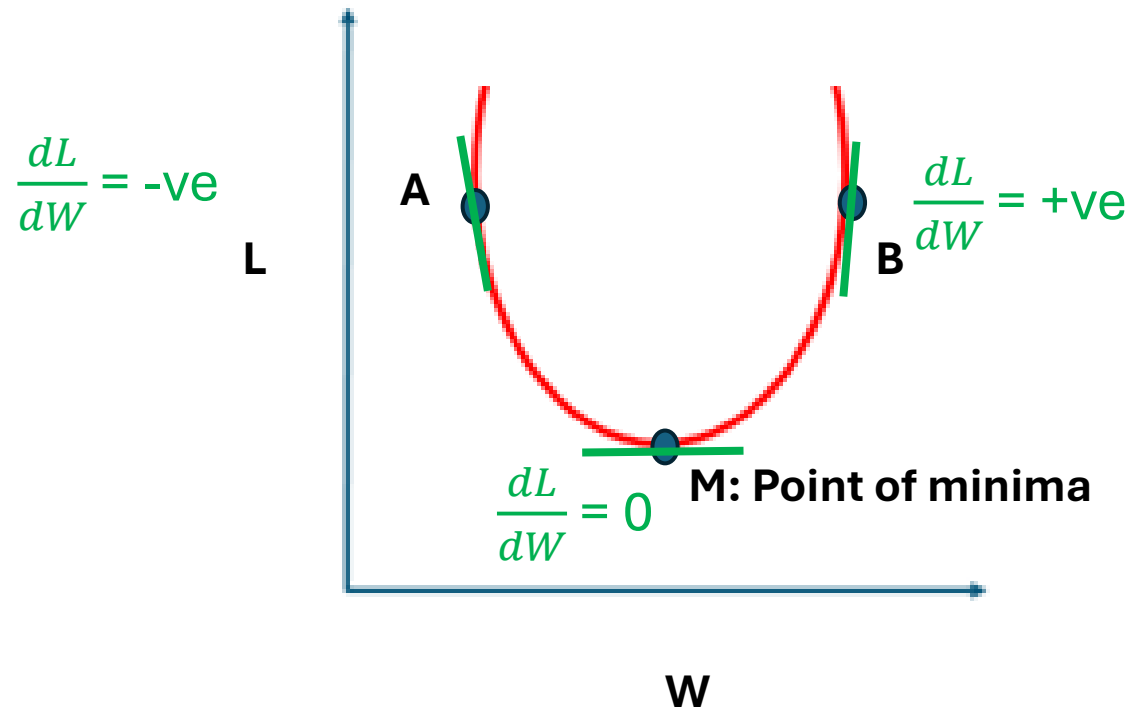
Gradient Descent

- Loss function is a function of all trainable parameters.
- We aim to determine how **changes in the weights W** affect the **loss L** . By adjusting **W in the appropriate direction**, we seek to **minimize the loss L** .

$$W_i^{\text{new}} = W_i^{\text{old}} - \eta * \frac{\partial E}{\partial W_i}$$

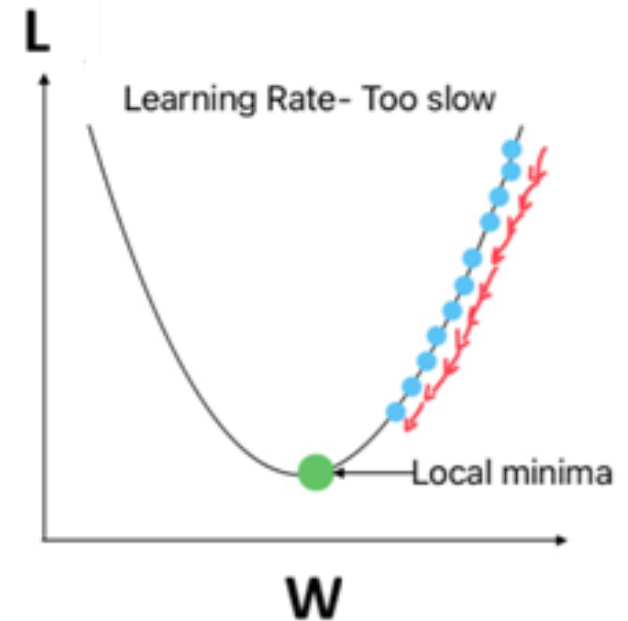
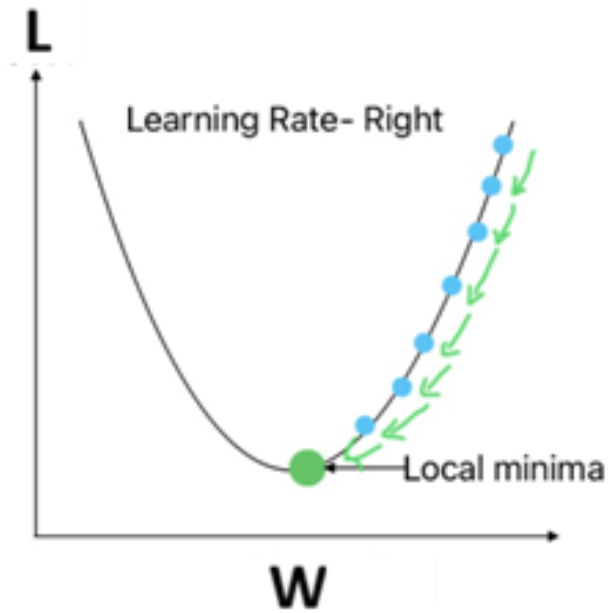
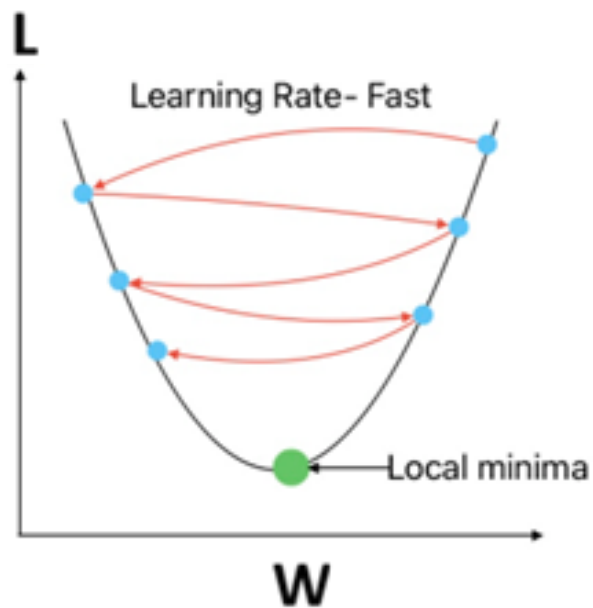
Gradient Descent

- Graph of L & W



$$W_i^{\text{new}} = W_i^{\text{old}} - \eta * \frac{\partial E}{\partial W_i}$$

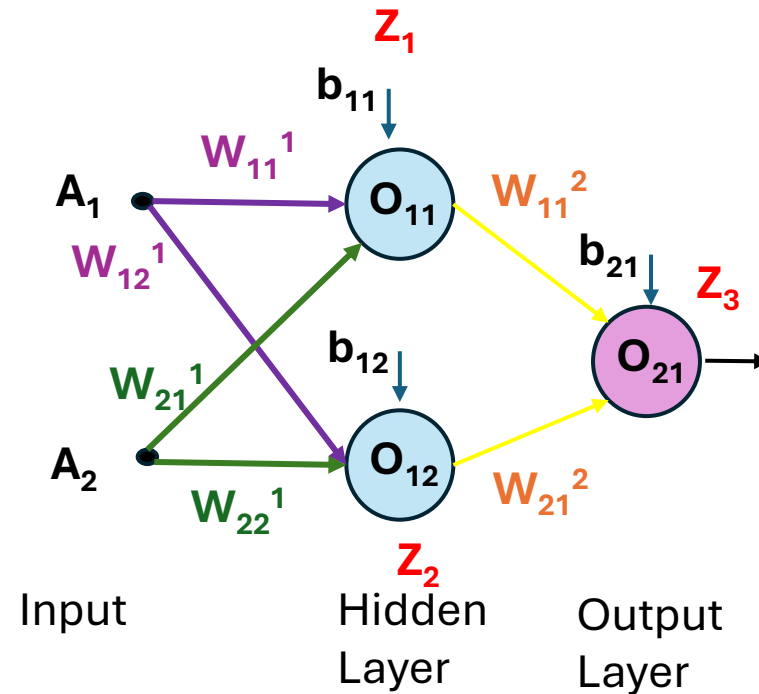
Impact of Learning rate



Multi-layer NN

$$W_i^{\text{new}} = W_i^{\text{old}} - \eta * \frac{\partial L}{\partial W_i}$$

	A ₁	A ₂	Labels
X ₁	X ₁₁	X ₁₂	L ₁
X ₂	X ₂₁	X ₂₂	L ₂
X ₃	X ₃₁	X ₃₂	L ₃
⋮	⋮	⋮	⋮
X _m	X _{m1}	X _{m2}	L _m



9 trainable parameters